

Universidade Federal do Rio Grande do Norte
Escola de Ciências e Tecnologia
IX Semana de Ciências e Tecnologia
Minicurso de Introdução a Banco de Dados

SQL e PostgreSQL

Parte 01

Breno Santana Santos
⟨breno.santos@ufrn.br⟩

10 de novembro de 2026

Sumário

1. PostgreSQL e DBeaver
2. Sistema de Segurança do PostgreSQL
3. Linguagem SQL
4. Gerenciamento de Banco de Dados e Tabelas
   - Linguagem DDL

PostgreSQL e DBeaver

- SGBD SQL de código aberto desenvolvido, distribuído e mantido por uma comunidade global de desenvolvedores voluntários.
- Contudo, várias organizações contribuem também para o projeto, a exemplo da EnterpriseDB, 2ndQuadrant e Pivotal.
- Principais características:
  - Software cuja licença PostgreSQL — similar a BSD e MIT;
  - Rápido;
  - Multitarefa;
  - Multiusuário;
  - SGBD SQL Robusto;
  - BD Relacional;
  - Escalável;
  - Fácil Usabilidade;
  - Trabalha em Cliente/Servidor;

PostgreSQL e DBeaver

- Principais características (cont.):
  - Escrito em C;
  - Testado com grande número de diferentes compiladores;
  - Suportado por vários SOs, ou seja, multiplataforma;
  - Suportado por várias linguagens de programação: C#, Java, Python, etc;
  - Sistema de privilégios e senha é muito flexível, seguro e diversificado, além de permitir a verificação baseada em *hosts*, por meio do arquivo `pg_hba.conf`;
  - Suporta os seguintes limites máximos:
    - $\approx 4,29$ bilhões de BDs;
    - Tabelas de 32 TB;
    - Linhas e Colunas de, respectivamente, 400 GB e 1 GB;
    - Tamanho de um BD e número de registros são ilimitados;
    - 250 a 1600 colunas por tabela — dependendo do tipo de dado;
  - Permite conexão TCP/IP.

PostgreSQL e DBeaver

- O DBeaver é uma ferramenta gráfica que permite trabalhar com diversos SGBDs.
- Principais funcionalidades:
  - **Desenvolvimento SQL:** criação e gerenciamento de conexões para diversos SGBDs. Permite executar comandos SQL por meio de um editor SQL;
  - **Diagramas de ER:** gera diagramas visuais automáticos das tabelas e seus relacionamentos dentro do banco de dados;
  - **Migração de Dados:** permite você migrar o seu banco de dados para o SQL Server, MariaDB e outros SGBDs.

Sistema de Segurança do PostgreSQL

- Possui um avançado sistema de segurança.
- Todo processo de segurança é configurado no arquivo `pg_hba.conf`.
- As senhas dos usuários são criptografadas por um algoritmo semelhante ao processo de autenticação de login do UNIX.
- A principal forma de gerenciamento de segurança é a concessão/revogação de permissões aos usuários que podem acessar o banco de dados.
- Caso necessite alterar as senhas dos usuários, veja o tutorial de segurança (Tutorial-Segurança-Postgresql.pdf).

Linguagem SQL

- Os SGBDs precisam fornecer linguagens para definir e manter um BD.
- Assim, foi desenvolvida a linguagem SQL, também conhecida como Linguagem de Consulta Estruturada — *Structured Query Language*.
- Desenvolvida pela IBM para ser a linguagem declarativa para manipular BDs.
- Tornou-se um padrão ANSI como linguagem padrão de manipulação de dados de um BD Relacional (BDR), apesar que algumas empresas criam dialetos específicos do SQL apenas disponíveis nos seus produtos (SGBDs).
- Permite manipular o BDR e seus objetos.

Linguagem SQL

- Dividida em três categorias:
  - Linguagem de Definição de Dados (DDL);
  - Linguagem de Manipulação de Dados (DML);
  - Linguagem de Controle de Dados (DCL).

- É um conjunto de comandos SQL que permite definir (criar), alterar ou remover objetos do BD, assim como o próprio BD. Em síntese, afeta a estrutura dos objetos do BD.
- Os principais comandos são:
  - Create;
  - Alter;
  - Drop;
  - Rename;
  - Truncate.

Linguagem SQL

Linguagem de Manipulação de Dados (DML)

- É um conjunto de comandos SQL que permite o acesso e manipulação dos dados de uma tabela em um BD.
- Pode ser dividida em:
  - **Alto Nível, Declarativa ou Não Procedural:** informa-se o que quer. Composta pelos comandos Insert, Update, Delete, Select, etc;
  - **Baixo Nível ou Procedural:** informa-se como quer. Composta por laços de repetições, condicionais, cursores, etc.

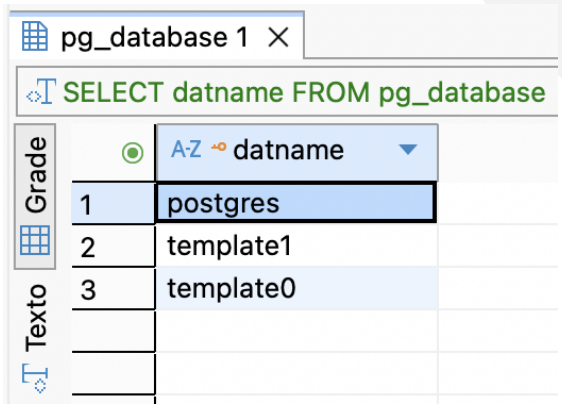
Linguagem SQL

Linguagem de Controle de Dados (DCL)

- É um conjunto de comandos SQL que permite o gerenciamento de usuários e permissões em um BD.
- Os principais comandos são:
  - Grant (concessão de permissões);
  - Revoke (revogação de permissões).
- Esse tipo de linguagem não será abordada no curso, somente as linguagens DDL e DML.

- Consiste na criação, manipulação e remoção de objetos e dados do BD, bem como essas operações relacionadas ao próprio BD.
- Criação de um Banco de Dados:
  - Diversos BDs separam as informações de domínios diferentes em um servidor de banco de dados.
  - **Sintaxe:** `CREATE DATABASE db_nome [OWNER usu_nome] [TEMPLATE db_nome_modelo] [ENCODING valor] [CONNECTION LIMIT valor];`
  - **Exemplo:**
    - `CREATE DATABASE sigaa ENCODING 'UTF8' CONNECTION LIMIT 1000;`
    - `CREATE DATABASE livraria OWNER zila_mamede;`
    - `CREATE DATABASE empresa OWNER dono TEMPLATE postgres;`

- Utilizar um BD como BD corrente ou *default* para uso:
  - **Sintaxe:** `\c db_nome;` (apenas no psql)
  - **Exemplo:** `\c sigaa;`
- Visualização/Listagem dos BDs contidos no PostgreSQL:
  - **Sintaxe:**
    - `SELECT datname FROM pg_database;`
    - `\l;` (apenas no psql)
  - As bases de dados postgres, template1 e template0 são BDs criados automaticamente pelo PostgreSQL para armazenamento de configurações.



- Alteração de um BD: permite alterar algumas das propriedades do BD já criado.
  - **Sintaxe:** `ALTER DATABASE db_nome [RENAME TO db_novo_nome] [OWNER TO novo_usuario] [CONNECTION LIMIT novo_limite];`
  - **Exemplo:**
    - `ALTER DATABASE sigaa RENAME TO bd_sigs CONNECTION LIMIT 500;`
    - `ALTER DATABASE livraria OWNER TO breno;`
- Remoção de um BD:
  - **Sintaxe:** `DROP DATABASE db_nome;`
  - **Exemplo:**
    - `DROP DATABASE sigaa;`
    - `DROP DATABASE livraria;`
    - `DROP DATABASE empresa;`

- Esquemas são fichários de organização de objetos criados em um BD.
- Criação de Esquemas:
  - **Sintaxe:** `CREATE SCHEMA sc_nome [AUTHORIZATION sc_usuario];`
  - **Exemplo:**
    - `CREATE SCHEMA graduacao;`
    - `CREATE SCHEMA graduacao AUTHORIZATION breno;`
- Listagem/Visualização dos esquemas de um BD:
  - **Sintaxe:** `\dn;` (apenas no psql)

- Alteração de Esquemas: possibilita a alteração do nome do esquema ou do seu proprietário.
  - **Sintaxe:**
    - **Alteração do nome:** `ALTER SCHEMA sc_nome RENAME TO sc_novo_nome;`
    - **Mudança de Proprietário:** `ALTER SCHEMA sc_nome OWNER TO {novo_dono_usuario | CURRENT_ROLE | CURRENT_USER | SESSION_USER};`
  - **Exemplo:**
    - `ALTER SCHEMA graduacao RENAME TO bacharel_cet;`
    - `ALTER SCHEMA graduacao OWNER TO breno;`
- Remoção de Esquemas: permite a remoção de um esquema no BD.
  - **Sintaxe:** `DROP SCHEMA sc_nome;`
  - **Exemplo:** `DROP SCHEMA graduacao;`

- Criação de Tabelas:
  - As tabelas dividem os dados de um BD a fim de agrupá-los segundo as suas correlações.
  - Toda coluna ou atributo possui um tipo de dado associado.
  - No PostgreSQL, temos vários tipos de dados definidos pelo sistema (pré-definidos).
  - Tipos de dados pré-definidos são:
    - **Inteiro:** `SMALLINT`, `INTEGER`/`INT`, `BIGINT`;
    - **Número de Precisão Arbitrária:** `DECIMAL`, `NUMERIC`;
    - **Ponto Flutuante:** `REAL`, `FLOAT`/`DOUBLE PRECISION`;
    - **Data e Hora:** `DATE`, `TIMESTAMP`, `TIME`;
    - **Caractere:** `CHAR`, `VARCHAR`, `BPCHAR`;
    - **Booleano:** `BOOL`/`BOOLEAN`;
    - **Textual:** `TEXT`.

- Criação de Tabelas (cont.):
  - Para o tipo de dado Número de Precisão Arbitrária, podemos informar a quantidade de dígitos (precisão, tamanho) e, caso seja necessário, o número de casas decimais (escala).
  - Para o tipo de dado Caractere, podemos informar a quantidade de caracteres (precisão, tamanho).
  - **Exemplos:**
    - `codigo INT`
    - `valor FLOAT`
    - `valor DECIMAL(10,2)`
    - `valor NUMERIC(10)`
    - `nome VARCHAR(200)`
    - `dt_nascimento DATE`
    - `horario TIMESTAMP`
    - `SEXO CHAR(1)`

- Criação de Tabelas (cont.):
  - Para criar tabelas, utiliza-se o comando `CREATE TABLE`.
    - É necessário definir os nomes das colunas e seus tipos de dados.
    - Definir as restrições para as colunas (`NOT NULL`, `NULL`, entre outros), se necessário.
    - Definir os valores padrões para as colunas, se necessário.
    - O tamanho máximo para a precisão do tipo Número de Precisão Arbitrária é 1.000. Por outro lado, sua escala varia de -1.000 a 1.000.

- Criação de Tabelas (cont.):
  - **Sintaxe:**

```
CREATE TABLE tb_nome (
    nome_atributo TIPO_DADO [RESTRICOES],
    ...
    nome_atributo TIPO_DADO [RESTRICOES]
);
```

- Criação de Tabelas (cont.):
  - **Exemplo:**

```
CREATE TABLE tb_aluno (
  matricula INT NOT NULL,
  nome VARCHAR(50) NOT NULL,
  endereco VARCHAR(50) NOT NULL,
  telefone VARCHAR(15) NULL,
  idade SMALLINT NOT NULL,
  altura DECIMAL(10,2) NOT NULL,
  dt_nascimento DATE DEFAULT NOW()
);
```

- Criação de Tabelas (cont.):
  - As colunas de uma tabela podem ter a propriedade de autoincremento, ou seja, o PostgreSQL atribui um valor automaticamente, permitindo que uma coluna seja incrementada a cada inserção de novos registros, identificando-os unicamente.
  - Existem três formas de definir o autoincremento:
    1. Uso do tipo de dado Serial (`SMALLSERIAL`, `SERIAL`, `BIGSERIAL`), o qual representa um tipo inteiro autoincremental;
    2. Uso do objeto `SEQUENCE`, definindo `nextval('tb_sequencia')` como valor padrão de uma coluna;
    3. Uso da cláusula `GENERATED [ALWAYS | BY DEFAULT] AS IDENTITY`.
  - As duas últimas opções são válidas apenas para o tipo de dado Inteiro.
  - Preferencialmente, somente uma coluna por tabela.
  - Geralmente, essa propriedade é definida para uma coluna que deve ser uma chave (`PRIMARY KEY` ou `UNIQUE KEY`).

- Criação de Tabelas (cont.):
  - **Exemplo:**

```
CREATE TABLE tb_aluno (
  matricula SERIAL NOT NULL,
  --- ou
  matricula INT NOT NULL DEFAULT nextval('
      seq_tb_aluno'),
  --- ou
  matricula INT NOT NULL GENERATED ALWAYS AS
      IDENTITY,
  ...
);
```

- Alteração de Tabelas:
  - Alteração da estrutura de uma tabela.
  - Permite:
    - Inclusão, modificação e remoção de colunas;
    - Inclusão e remoção de restrições;
    - Alteração do identificador (nome) da tabela.
  - Especificações:
    - **Inclusão de Coluna:** `ADD COLUMN nome_atrib tipo_dado [RESTRIÇÕES];`
    - **Renomeação de Coluna:** `RENAME COLUMN antigo_nome_atrib TO novo_nome_atrib;`
    - **Alteração do Tipo de Dado de Coluna:** `ALTER COLUMN nome_atrib SET DATA TYPE novo_tipo_dado;`

- Alteração de Tabelas (cont.):
  - Especificações (cont.):
    - **Remoção de Coluna:** `DROP COLUMN nome_atrib;`
    - **Renomeação de Tabela:** `RENAME TO tb_novo_nome;`
  - **Exemplo:**

```
ALTER TABLE tb_aluno ADD COLUMN
    nm_responsavel VARCHAR(50) NOT NULL;
ALTER TABLE tb_aluno ADD COLUMN valor_mensal
    NUMERIC(10,2) NOT NULL, situacao CHAR(1);
ALTER TABLE tb_aluno RENAME COLUMN telefone
    TO fone_responsavel;
```

- Alteração de Tabelas (cont.):
  - **Exemplo:**

```
ALTER TABLE tb_aluno ALTER COLUMN matricula
    SET DATA TYPE BIGINT;
ALTER TABLE tb_aluno DROP COLUMN ALTURA;
ALTER TABLE tb_aluno RENAME TO tb_aluno_grad;
```

- Remoção de Tabelas: exclui a tabela do BD corrente.
  - **Sintaxe:** `DROP TABLE tb_nome;`
  - **Exemplo:** `DROP TABLE tb_aluno;`
- Eliminação dos Registros de uma Tabela: esvazia uma tabela do BD corrente, ou seja, remove todos os registros.
  - **Sintaxe:** `TRUNCATE TABLE tb_nome;`
  - **Exemplo:** `TRUNCATE TABLE tb_aluno;`

- Listagem de Tabelas de um BD (apenas no psql):
  - **Sintaxe:**
    - `\dt;`
    - `\dt+;`
  - **Exemplo:**

```
postgres=# \dt
            List of tables
 Schema |   Name   | Type  |  Owner
--------+----------+-------+----------
 public | tb_aluno | table | postgres
(1 row)

postgres=# \dt+
                                      List of tables
 Schema |   Name   | Type  |  Owner   | Persistence | Access method |  Size   | Description
--------+----------+-------+----------+-------------+---------------+---------+-------------
 public | tb_aluno | table | postgres | permanent   | heap          | 0 bytes |
(1 row)
```

- Visualização da Estrutura de Tabelas (apenas no psql):
  - Permite visualizar a estrutura de uma tabela: colunas, tipos de dados e restrições.
  - **Sintaxe:**
    - `\d nome_tabela;`
    - `\d+ nome_tabela;`
  - **Exemplo:**

```
postgres=# \d tb_aluno
                                        Table "public.tb_aluno"
    Column     |          Type          | Collation | Nullable |                    Default
---------------+------------------------+-----------+----------+-----------------------------------------------
 matricula     | integer                |           | not null | nextval('tb_aluno_matricula_seq'::regclass)
 nome          | character varying(100) |           | not null |
 fone          | integer                |           |          |
 idade         | smallint               |           | not null |
 dt_nascimento | date                   |           | not null |
Indexes:
    "tb_aluno_pkey" PRIMARY KEY, btree (matricula)
```